跨端页面秒开，腾讯开源框架Kuikly是怎么做到的？

原创 腾讯程序员 腾讯技术工程 2026年6月10日 02:37 广东 108人

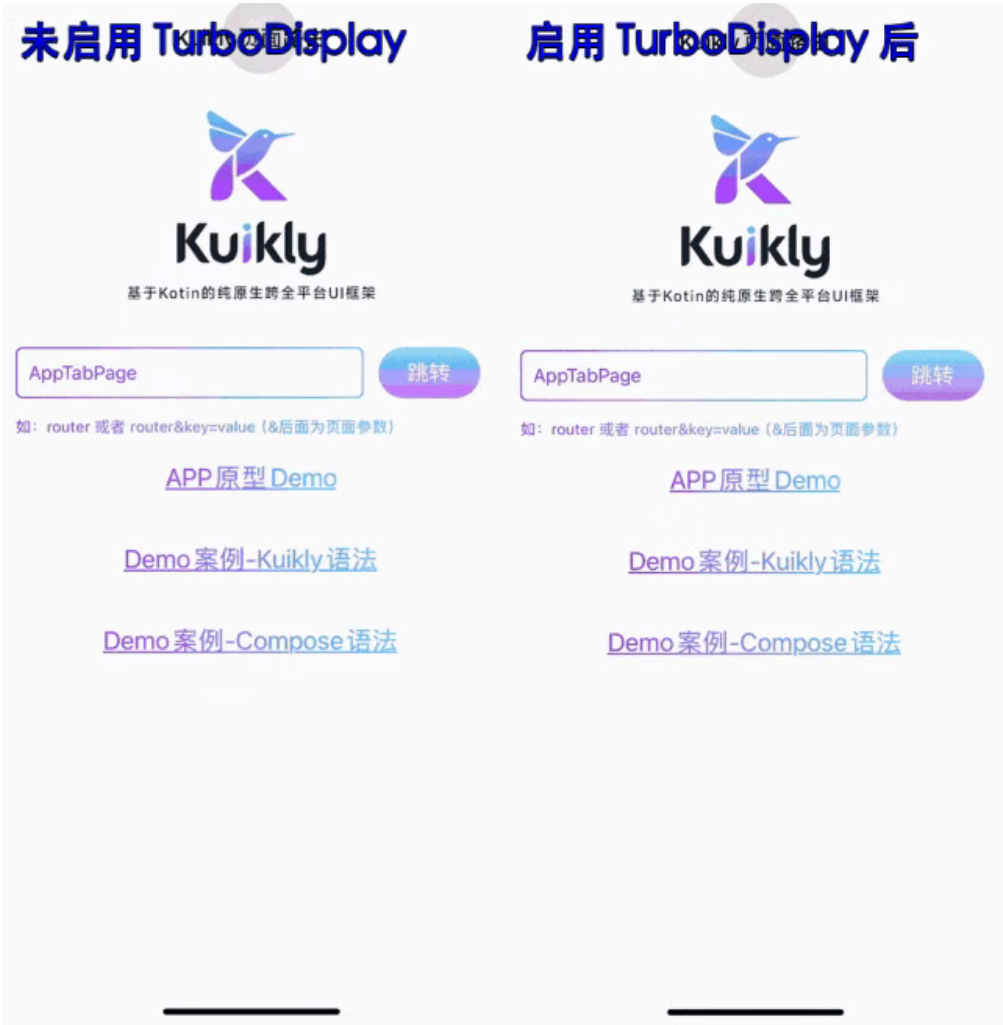


Kuikly 是腾讯基于 Kotlin Multiplatform (KMP) 技术构建的全平台高性能开发框架，具有一码多端、极致易用、动态灵活特点，覆盖 Android、iOS、HarmonyOS、H5、微信小程序、Mac 六大平台。

得益于 Kotlin/Native 编译产物与原生渲染管线，Kuikly 已具备原生级性能表现；为进一步压榨首屏体验，我们推出了加速方案 TurboDisplay，通过节点缓存、主线程端侧直出与双线程并行渲染，在二次打开场景下实现"秒开"，并在 QQ 游戏、输入法、腾讯地图等线上业务中取得 60%~80% 以上的耗时优化。本文将深入解析其设计思路与核心机制。

一、运行效果展示

TurboDisplay 是 Kuikly 自研的首屏加速方案，在页面二次打开场景下实现了"秒开"。运行效果如下：



二、为什么要做首屏加速

页面打开耗时是移动客户端长期面临的共性体验问题——从用户点击入口到首屏呈现，链路上要依次经历「容器初始化、运行时与渲染引擎准备、业务代码执行、数据请求与解析、布局测量、节点构建与最终上屏」，任何一环的耗时累加都直接转化为可感知的等待。

这种耗时并非某一类技术栈的专属问题，即便纯原生开发也无法回避**业务复杂度、网络波动、设备性能差异**等客观因素。

Kuikly 虽然通过 Kotlin/Native 编译产物与原生渲染管线获得了原生级执行性能，但同样走在这条串行链路上，这意味着单纯优化"代码执行更快"已逼近收益上限。若想从根本上加速首屏显示，必须跳出执行效率本身，从渲染链路的调度方式上寻找新的突破口。

三、传统方案为什么都不够

围绕首屏加速，业界已经积累了不少通用优化手段。它们从不同链路切入，在各自适用的场景下都能带来一定收益，但无论是原生开发还是跨端框架，都难以做到"秒开"体验。

方案	能解决什么	主要问题	为什么不够支撑秒开
Bitmap 截图占位	立即给出视觉反馈	不可交互，无法参与真实滚动与状态恢复	解决的是“看起来有内容”，不是“真实页面已可用”
接口数据缓存	缩短等网络数据的时间	不覆盖本地业务执行、节点构建、布局测量	数据就绪不等于页面就绪
骨架屏 / Loading	缓解等待焦虑	本质是等待过程的包装	没有缩短首屏可交互时间

因此要真正实现 "打开即可见、可交互"，缓存的对象不能只是图像或数据，而应该深入到框架自身的渲染产物 —— **缓存构建好的渲染节点**，从而在下一次打开时跳过中间环节，直接以可交互的形态秒级呈现。**那么Kuikly 在这条路上应该怎么走？**

四、方案选型

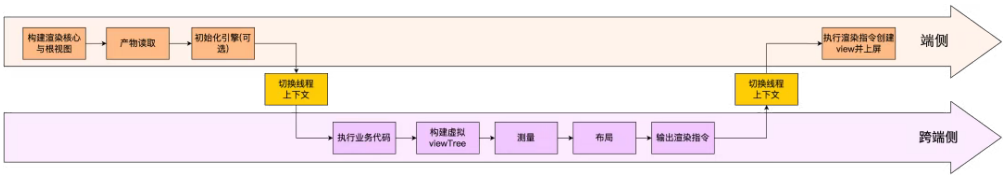
4.1 好的首屏加速方案是什么样

理想首屏缓存加速方案，应同时满足**业务无侵入、渲染及时且正确**的要求：不依赖业务的具体实现，且业务接入成本尽可能小，最终呈现的页面也是与业务真实逻辑严格对应。

这三者共同决定了缓存加速方案是"普惠加速"还是"个例 trick"。业务无侵入决定它能不能规模化铺开，及时性决定用户看到的是不是最新页面，正确性决定它在生产环境能不能被信任。

4.2 最容易想到的方案：节点直出，够用吗？

基于"缓存渲染节点 "这一思路，最自然的做法就是节点直出，把渲染节点的结构、属性、样式持久化，下次打开时读出并直接重建视图，跳过"执行业务代码 → 布局测量 → 节点构建"这一段。在 Kuikly 中，页面最终上屏时会被抽象为一组低阶渲染节点与基本渲染指令，节点直出能自然适配这套管线。



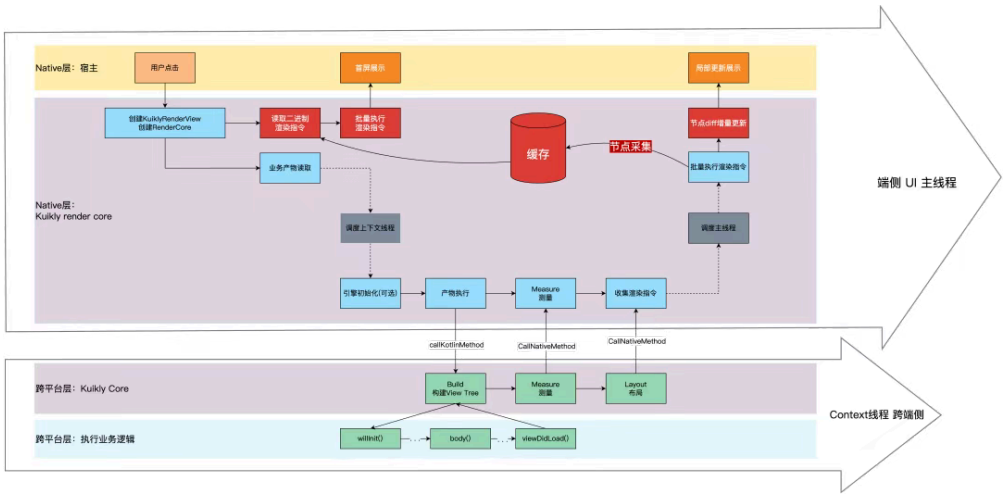
但节点直出只覆盖了跨端侧的业务执行与节点构建等阶段，前置的引擎初始化操作仍然无法省略，仍会延缓首屏展示。所以直接套用节点直出方案，距离"页面打开即可见、可交互"仍有明显差距。要再往前走，必须对节点直出方案本身做根本性的改造。

4.3 最终方案：把缓存恢复从主链路中拆出

节点直出只是缩短了链路，并未重构链路。缓存恢复仍嵌套在业务逻辑执行的链路中。这意味着首屏加速不再只是“把原链路做快一点”，而是要把缓存恢复从原链路中拆出来，形成一条独立执行路径。而实现此路径，还需要进一步解决三个关键问题，如何让缓存恢复提前发生，缓存内容如何持续更新，以及缓存首屏如何与真实页面实现平滑衔接。

五、方案设计

围绕这三个问题，TurboDisplay 的核心设计可以拆成三部分：双线并行、节点采集和增量更新。前两者解决“如何更早展示”和“展示什么”，后者解决“如何在不中断交互的前提下切回真实页面”。此外，为了适配Kuikly的页面执行链路，TurboDisplay还设置了缓存一致性、兜底写入等机制，提供了强制刷新、节点树结构捕捉等业务控制项，以及面向特殊场景的扩展拓展。受篇幅所限，下文不展开全部细节，将聚焦三个核心机制以及特殊场景的拓展支持，阐明 Kuikly 的节点直出首屏加速方案。



5.1 设计骨架：三项核心机制

先看Kuikly是如何通过双线并行、节点采集以及增量更新实现以缓存恢复成为独立快速路径的首屏加速方案。

5.1.1 双线并行

在Kuikly实施“缓存恢复成为独立快速路径”，是把“页面的完整执行链路”内构建首屏的执行过程搬到端侧提前多执行一遍：

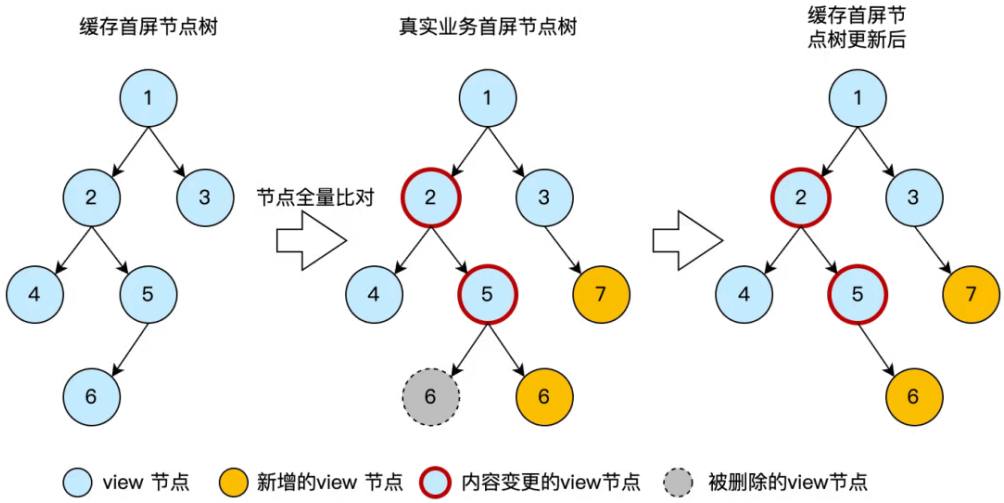
- 在端侧承载容器以及根view构建完成后，就直接读取本地缓存并转换为节点树，产生渲染指令让首屏快速在端侧上屏；

- 而原本的执行链路始终保持不变，端侧进行线程调度后，在跨端侧仍按正常流程在后台执行这一轮真实页面的业务逻辑、布局测量与节点构建，再产生批量的渲染指令并生成一颗端侧的真实页面节点树，等待后续与缓存节点树比对差异，更新实际的首屏渲染内容。

5.1.2 节点采集

TurboDisplay所存储节点是端侧节点树上节点的完整内容，包括每个节点的属性、事件、布局frame、Shadow 信息、节点上的方法调用以及节点树结构的变化，而在采集时，TurboDisplay能够系统地比较缓存首屏与真实页面节点树在树形、节点内容之间的差异，将差异更新至缓存节点树，然后自动、周期限频地把缓存节点树写盘。

此外，节点采集还支持业务自定义是否需捕捉节点树之间的结构差异，若未开启此能力，则缓存的始终是业务真实首屏所对应的节点及其最新的内容。



5.1.3 增量更新

TurboDisplay在快速直出首屏内容之外，最重要的就是如何实现真实页面逻辑平稳的在端侧已展示的缓存首屏上展现出来，为避免业务真实逻辑与缓存首屏之间存在数据差异，进而造成实际渲染时出现闪屏和跳变，局部更新的方式是其中最稳定的解决方式。

通过跨端侧调度主线程，端侧批量执行渲染指令建立业务真实首屏对应的节点树，然后执行与缓存节点树之间的属性、布局frame、调用方法的参数此些内容的差异比较，全量的回放首屏过程中所记录的交互事件，以及直接新建此前不存在的节点，从而在不抖动屏幕内容的情况下，局部的增量更新上业务的实际首屏内容，实现缓存首屏与业务真实首屏之间的平滑切换。

5.2 拓展支持：状态恢复与事件保序

三项核心机制支持了节点直出快速显示Kuikly页面首屏，但真实页面并不总停留在初始位置。缓存首屏展示后，用户可能已经发生交互，列表页也可能需要恢复到上一次访问位置，这会让端侧已展示的内容与跨端侧后台构建出的默认首屏产生错位，这时就需要借助diff来完成修正。但执行原有的diff后，却出现了首屏跳变回默认内容，首屏内容落在屏幕之外而出现白屏。

5.2.1为何 diff 增量更新会成为问题

TurboDisplay 的"端侧直出 + 跨端侧后台构建"并行设计，意味着缓存首屏比真实业务首屏出现得更早——这个"更早"，正是问题的根源。

端侧直出后，无论是恢复到历史位置还是响应用户交互，端侧都已经呈现了非默认状态；但跨端侧构建的真实页面仍处于默认位置，尚未感知这些变化。当真实页面通过 diff 在端侧准备展示时，对比的是"端侧已恢复/交互后的状态"与"真实页面默认状态"，差异被全量更新，导致端侧已呈现的内容被覆盖回默认效果，表现为位置跳变或交互结果丢失。

要解决这一覆盖问题，关键在于落在了首屏内容需变化时，diff 何时执行才正确。

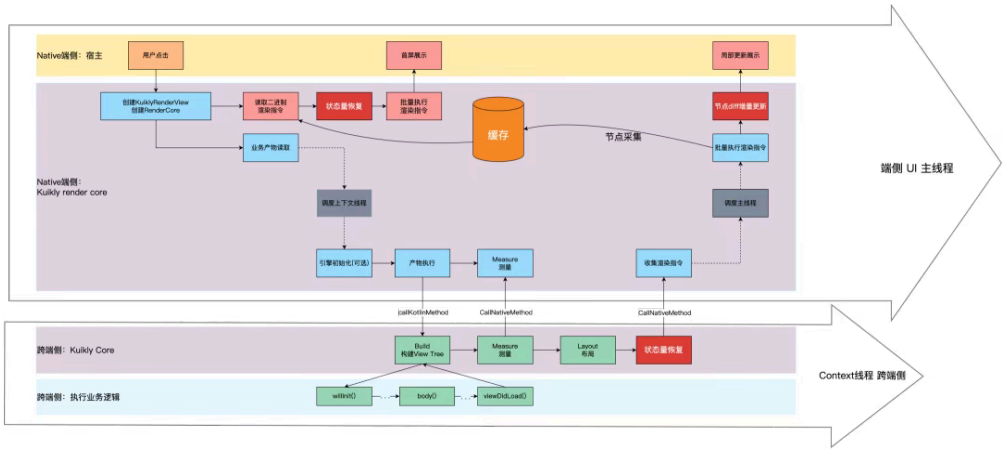
5.2.2解决方案：由跨端侧调度diff

为了找到正确的 diff 时机，我们先后尝试了两种方案：

方案	做法	失败原因	根因
固定延时	端侧直出后延迟 T ms 再执行	固定等待时间无法适配跨端侧的不确定耗时：短了 Diff 还没完，恢复仍被覆盖；长了用户先看到默认位置，体验反而劣化	固定延时试图用猜测覆盖不确定的时序
监听端侧渲染完成	端侧首屏视图构建完成后触发 diff 执行	端侧完成 ≠ 跨端侧完成，Diff 仍在恢复之后到达并覆盖	端侧信号无法作为跨端侧进度的参照

两次尝试的失败指向同一个根因：端侧无法预知跨端侧的真实页面何时构建完成，无论用延时还是用本地信号，都无法在端侧找到一个稳定的执行时机。这个根因反过来也给出了方向——**既然端侧无法判断，那就让拥有完整时序信息的跨端侧来负责调度恢复。**

具体做法是，将 Diff 的执行时机从"真实页面到达端侧即执行"延后为"恢复内容生效后再执行"，也就是延迟 Diff。具体而言，跨端侧在构建真实节点树时精确感知自身进度，在生成渲染指令前先执行恢复逻辑：将滚动偏移量应用到真实页面节点上、回放缓存首屏期间暂存的交互事件使跨端侧先响应，交付给端侧的真实首屏本身已处于恢复后的状态。此时 diff 对比的两端基础一致，非默认内容不再是差异项，自然不会被覆盖。下图即是由跨端侧执行调度后的首屏加载过程，其中**红色矩形部分即页面恢复所引入的新变化。**



在解决了 diff 时机的问题后，无论是业务逻辑默认首屏内容，还是交互之后的动态内容，TurboDisplay 都能实现稳定且正确的展示。

六、性能表现与验证

为了验证 TurboDisplay 的实际效果，我们分别从实验室环境与线上真实业务场景两个维度进行了性能测试。结果一致表明，TurboDisplay 在二次打开场景中能够显著加速首屏的显示，且在复杂页面中保持稳定的优化收益。

6.1 实验室对比

我们在 iPhone 13 Pro Max 和iPhone 7 Plus 两种设备上进行了测试，测试场景为页面二次打开。以下 GIF 中第 1 帧做了延迟 100 毫秒处理，便于观察是否实现首帧即现。

- iPhone 7 Plus 对比



- iPhone 13 Pro Max 对比



6.2 线上数据验证

TuroDisplay 已经在QQ 游戏、输入法以及腾讯地图等诸多业务场景上线，并取得了65-80%的耗时优化幅度，以下为部分页面实际线上对比数据：（注：优化前页面的代码与数据均已本地缓存，启动过程无网络耗时，均为逻本地辑耗时）

页面（单位：ms）	首屏耗时 – 优化前	首屏耗时 – 优化后(启用TurboDisplay)	耗时优化幅度
腾讯地图业务 – 页面 A	281	48	82.9%
QQ 游戏业务 – 页面 B	146	40	72.6%
QQ 游戏业务 – 页面 C	116	32	72.4%
输入法业务 – 页面 A	126	42	66.7%
输入法业务 – 页面 B	54	17	66.7%

七、如何体验

TurboDisplay 在设计之初便考虑了业务接入的轻量化。实际使用中，业务侧也仅需实现开关接口 TurboDisplayKey，即可为 Kuikly 页面开启首屏加速能力，典型的接入代码如下：

```
// Native端侧页面容器 KuiklyRenderViewController.m
// 实现TurboDisplay开关接口
- (NSString *)turboDisplayKey {
    return pageName;
}
```

其他业务控制项的使用方式及相关细节，可参考 官方文档 作进一步了解。

八、后续规划

当前提供的 TurboDisplay 方案主要聚焦于页面二次打开场景优化，针对首次打开场景，团队内部也正在进行方案设计和开发，核心思路：

- 在编译期对首屏逻辑进行静态分析与提取
- 在运行时通过微指令引擎快速展示首屏
- 再与原产物执行链路平滑衔接

欢迎大家保持关注。

九、关于Kuikly

当前Kuikly已经开源，有兴趣和有需要的产品，可以通过以下方式访问 Kuikly 仓库和文档，欢迎Star、Watch与体验：

👉 Github 仓库 | 📚 官方文档

Kuikly框架属于腾讯端服务联盟（tds.qq.com）的重要成员，欢迎关注及了解更多信息：

- 腾讯端服务官网： TDS 腾讯端服务
- TDS Framework官网： 跨平台框架

「腾讯端服务」服务号	「TDS Framework」公众号	在线咨询





阅读 1.2万

留言

写留言

还没有留言，来写首评